VERIQTA

Real Outages.
Real Lessons.
Production Ready.

KUBERNETES PRODUCTION OUTAGES

EVERY ENGINEER SHOULD KNOW

VOLUME 2

NETWORKING & SERVICE FAILURES

- SERVICE DISCOVERY
- CLUSTER NETWORKING
- DNS FAILURES
- INGRESS & TLS ISSUES
- LOAD BALANCING
- TRAFFIC INVESTIGATION



Focus Areas

- ☑ DNS & CoreDNS Issues
- ☑ Ingress & TLS Failures
- ☑ NetworkPolicy Problems
- ☑ Load Balancer Outages
- ☑ CNI & Pod Connectivity
- ☑ Service Mesh Failures
- ☑ Real Production Examples

PRACTICAL INVESTIGATION
Step-by-step troubleshooting workflows

SENIOR LEVEL COMMANDS
Real commands used in production incidents

REAL OUTAGES REAL LESSONS
Learn from real failures and how they were solved

PRODUCTION READY
Build the skills that keep systems running

TROUBLESHOOT SMARTER. RESTORE FASTER. PREVENT NEXT TIME.

Instagram: @veriqta LinkedIn: VERIQTA X: @veriqta YouTube: VERIQTA

VERIQTA

HOW KUBERNETES NETWORKING ACTUALLY WORKS

Networking is the foundation of everything in Kubernetes.

1. POD-TO-POD COMMUNICATION



- All Pods have flat, routable IPs.
- Pods can communicate directly across nodes.
- No NAT between Pods.
- Built by your CNI plugin.

2. SERVICE ABSTRACTION



- A Service gives Pods a stable virtual IP and DNS name.
- It abstracts dynamic Pods.
- Traffic is load balanced to healthy endpoints.

3. KUBE-PROXY RESPONSIBILITIES



- Watches Services and Endpoints.
- Programs iptables/IPVS rules.
- Routes traffic to the correct Pods.
- Ensures Service VIPs work on every node.

4. CNI PLUGINS (NETWORK BACKBONE)



CNI gives every Pod an IP and connectivity.

Popular CNI plugins:

- Calico
- Cilium
- Flannel
- Weave Net

5. KUBERNETES SERVICE TYPES

CLUSTERIP (Default)



10.96.x.x

- Internal access only
- Accessible within the cluster

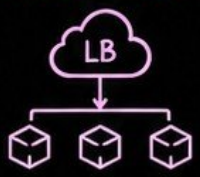
NODEPORT



Node IP:30080

- Exposes Service on each node IP
- Accessible from outside the cluster

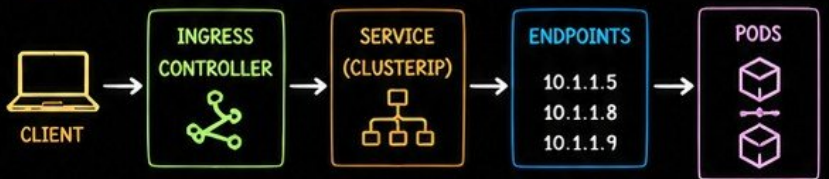
LOADBALANCER



- Creates external Load Balancer
- Cloud provider provisions it

The same Service can be exposed using different types depending on your use case.

6. END-TO-END TRAFFIC FLOW (DEEP DIVE)



1. Client sends request to your app.
2. Ingress applies rules (host, path, TLS) and routes the request.
3. Service selects healthy endpoints.
4. Traffic is load balanced to one of the Pods.
5. Pod processes the request and sends response back.

Responses travel back the same path in reverse.

7. PRODUCTION WORKFLOW (THE GOLDEN PATH)



CLIENT
Users / Systems make requests.

INGRESS
Entry point, routing, TLS termination, security.

SERVICE
Stable virtual IP and load balancing to endpoints.

ENDPOINT (POD IPs)
List of healthy Pod IPs behind the Service.

POD
Application handles the request and returns response.

KEY TAKEAWAY

Kubernetes networking is a combination of CNI, kube-proxy, Services, and Ingress working together to move traffic reliably and securely.

REMEMBER: When something is not reachable, always follow the traffic path step-by-step. Don't guess. Validate each hop.

VERIQTA

SERVICE HAS NO ENDPOINTS

A Service with no endpoints cannot send traffic to any Pod. Users will experience failures.

A Kubernetes Service is only as healthy as its Endpoints.
No Endpoints = No Traffic = Outage

1. COMMON CAUSES

LABEL SELECTOR MISMATCH
Service selector does not match Pod labels. No Pods are selected.

PODS NOT READY
Pods exist but are not Ready. Readiness probe failing.

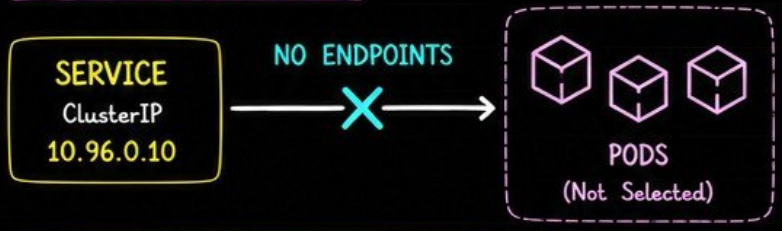
WRONG NAMESPACE
Service and Pods are in different namespaces.

FAILED ROLLOUT
New Pods never became Ready. Old Pods scaled down.

ENDPOINTSLICE ISSUES
EndpointSlice controller not working or corrupted data.

INCORRECT SERVICE CONFIGURATION
Wrong ports, targetPort, or selector. Service type misconfiguration.

2. WHAT IT LOOKS LIKE



SYMPTOMS
- kubectl get endpoints shows <none>
- No traffic reaches the Pods
- 503 / connection errors
- Application timeouts

3. QUICK VISUAL CHECK

CHECK	EXPECTED	PROBLEM
Service Selector	Matches Pod Labels	No match
Pods Ready	READY = 1/1	0/1 or NotReady
Namespace	Same Namespace	Different NS
Endpoints	Has IPs	<none>
EndpointSlice	Has Endpoints	Empty or Missing

4. SENIOR INVESTIGATION COMMANDS

`kubectl get svc`	List all Services and check the CLUSTER-IP and PORT. Verify the Service exists in the correct namespace.
`kubectl get endpoints`	Show Endpoints for all Services. Look for <none> in the ENDPOINTS column.
`kubectl get endpointslices`	Show EndpointSlice resources. Verify endpoints are present and up-to-date.
`kubectl describe service <service>`	Deep dive into the Service configuration. Check Selector, Ports, Events, and Warnings.

PRO TIP: Always validate Service selector against actual Pod labels. Most "No Endpoints" issues are due to simple mismatches.

REMEMBER
No Endpoints = No Traffic.
Fix the root cause, not the symptom.

VERIQTA COREDNS FAILURE

CoreDNS is the DNS service for your cluster. If DNS breaks, almost everything breaks.

CoreDNS is responsible for resolving all DNS queries inside the cluster. When CoreDNS fails, Pods cannot resolve Services or external domains.

1. COMMON CAUSES

COREDNS CRASH
CoreDNS Pods crash due to bugs, panics, or resource issues.

DNS LOOP
Misconfigured forwarders or plugins cause infinite loops.

MEMORY EXHAUSTION
High query load causes CoreDNS to run out of memory and restart.

UPSTREAM DNS UNAVAILABLE
CoreDNS cannot reach external DNS servers to resolve queries.

MISCONFIGURED COREFILE
Syntax errors, wrong upstreams, or missing plugins break DNS.

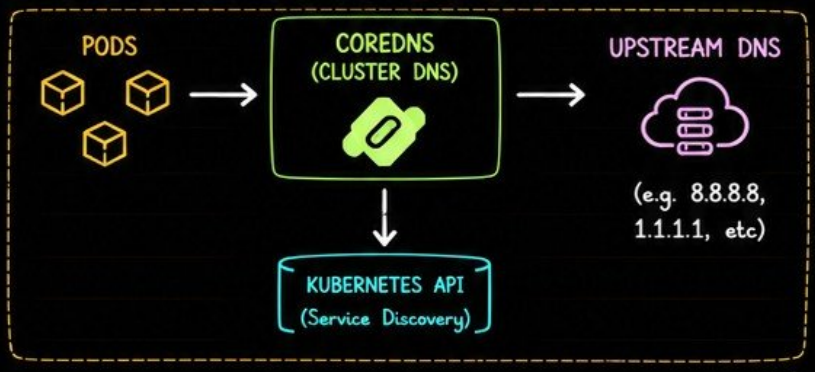
NETWORK FAILURE
Network policies, CNI issues, or connectivity problems block DNS.

2. WHAT IT LOOKS LIKE

- Pods cannot resolve Service names
- Pods cannot resolve external domains
- Applications show timeouts or connection errors
- Everything appears as application failure

But the real problem is DNS

3. COREDNS ARCHITECTURE (OVERVIEW)



4. SENIOR INVESTIGATION COMMANDS

`kubectl -n kube-system get pods -l k8s-app=kube-dns`	Check CoreDNS Pod status.
`kubectl -n kube-system logs deploy/coredns`	Check CoreDNS logs for errors.
`kubectl -n kube-system describe pod <coredns-pod>`	Inspect Pod events and restart reasons.
`kubectl -n kube-system get configmap coredns -o yaml`	Exec into CoreDNS Pod for deeper checks.
	Review the Corefile configuration.

5. PRODUCTION INSIGHT

Most "application failures" are actually DNS failures.

- ☑ Always test DNS first during any outage.
- ☑ Check CoreDNS before assuming the app is broken.
- ☑ Validate both internal and external name resolution.

BEST PRACTICE

- ☑ Run multiple CoreDNS replicas
- ☑ Set resource requests/limits
- ☑ Monitor both latency & errors
- ☑ Test DNS during deployments

PODS CANNOT RESOLVE DNS

If DNS is broken, Pods are isolated from the world.

When Pods cannot resolve DNS, Services, external APIs, databases, and everything depending on DNS will fail.

1. FAILURE SCENARIOS

SERVICE NAMES FAIL
Pods cannot resolve Kubernetes Service names (e.g. myapp.default.svc).

EXTERNAL DOMAINS FAIL
Pods cannot resolve external domains (e.g. google.com, api.example.com).

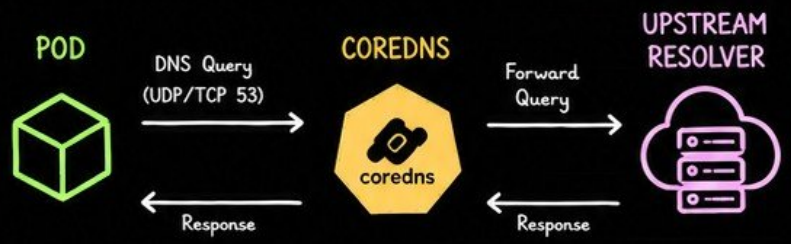
SEARCH DOMAINS INCORRECT
Wrong search domains cause Pods to query the wrong DNS names.

NDOTS CONFIGURATION
Incorrect ndots value causes slow or incorrect DNS lookups.

DNS TIMEOUT
DNS queries timeout due to slow CoreDNS or upstream issues.

NETWORKPOLICY BLOCKING DNS
NetworkPolicy prevents Pods from reaching CoreDNS (UDP/TCP 53).

2. WORKFLOW: HOW DNS RESOLUTION WORKS



1. Pod sends DNS query to CoreDNS service (usually 10.96.0.10).*
2. CoreDNS checks cache, forwards query to upstream resolver if needed.
3. Upstream resolver returns the answer back to CoreDNS, then to the Pod.

3. COMMON DNS QUERY FLOW INSIDE A POD

/etc/resolv.conf (example)

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

TIP: High ndots values cause Pods to first try the full search domains before the actual domain. This slows down resolution.

4. SENIOR INVESTIGATION COMMANDS

`kubectl exec -it <pod> -- cat /etc/resolv.conf`	Check DNS configuration inside the Pod.
`kubectl exec -it <pod> -- nslookup kubernetes.default`	Test Service name resolution.
`kubectl exec -it <pod> -- nslookup google.com`	Test external domain resolution.
`kubectl -n kube-system get pods -l k8s-app=kube-dns`	Check CoreDNS Pod status.
`kubectl -n kube-system logs deploy/coredns`	Inspect CoreDNS logs for errors.
`kubectl get networkpolicy -A`	Check for NetworkPolicies blocking DNS.

5. QUICK CHECKLIST

- ☑ CoreDNS Pods are Running and Ready
- ☑ Pod can reach 10.96.0.10 on UDP/TCP 53
- ☑ Service name resolution works
- ☑ External DNS resolution works
- ☑ Upstream DNS servers are reachable
- ☑ resolv.conf is correct
- ☑ NetworkPolicies allow DNS traffic
- ☑ No DNS timeouts or errors in logs

REMEMBER

DNS is the first dependency of almost every application. Fix DNS first, then move up the stack.

PRODUCTION INSIGHT: Never debug the application before confirming DNS works. Fix DNS, and half of your outages will disappear.

VERIQTA

INGRESS NOT ROUTING TRAFFIC

Ingress is the gateway to your applications. If it is broken, users see nothing.

If the Ingress is misconfigured or unhealthy, traffic from the world will never reach your Services or Pods.

1. COMMON CAUSES

WRONG INGRESSCLASS
IngressClass in the Ingress does not match the controller watching the cluster.

MISSING BACKEND SERVICE
Ingress points to a Service that does not exist or is in the wrong namespace.

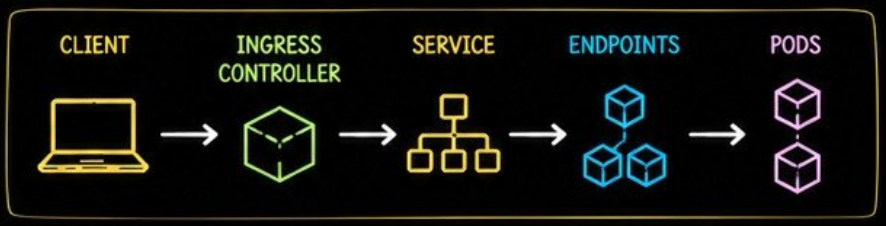
INVALID HOST RULE
Host in the Ingress does not match the request Host header (DNS issue or incorrect rule).

PATH MISMATCH
Request path does not match any configured path rule in the Ingress.

CONTROLLER UNAVAILABLE
Ingress controller Pods are down, crashing, or not ready.

TLS MISCONFIGURATION
TLS certificate missing, wrong Secret, expired cert, or hostname mismatch.

2. TRAFFIC FLOW (WHAT SHOULD HAPPEN)



3. HOW TO VERIFY

`kubectl get ingress -A`	Check Ingress exists and correct namespace.
`kubectl describe ingress <ingress> -n <ns>`	Check Events, Rules, Backend, and Class.
`kubectl get ingressclass`	Verify the IngressClass is present.
`kubectl get svc -n <ns>`	Verify backend Service exists and is healthy.
`kubectl get endpoints -n <ns>`	Verify Service has healthy Endpoints.
`kubectl get pods -n <ingress-ns>`	Check Ingress controller Pods are Running.
`kubectl logs -n <ingress-ns> deploy/<controller>`	Check controller logs for routing errors.

4. SYMPTOMS YOU WILL SEE

- 404 Not Found
- 502 Bad Gateway
- 503 Service Unavailable
- 504 Gateway Timeout
- Connection refused
- TLS handshake errors

Symptoms usually look like application issues, but Ingress is often the root cause.

5. QUICK CHECK BY TYPE

TYPE	CHECK	COMMAND
IngressClass	Is the Class correct?	`kubectl get ingressclass`
Backend Service	Does the Service exist?	`kubectl get svc -n <ns>`
Endpoints	Does the Service have Endpoints?	`kubectl get endpoints -n <ns>`
Host Rule	Does the Host match request?	`kubectl describe ingress`
Path Rule	Does the Path match?	`kubectl describe ingress`
Controller	Are Controller Pods Ready?	`kubectl get pods -n <ingress-ns>`
TLS	Is the TLS Secret valid?	`kubectl describe secret <tls-secret>`

6. PYMDUCTION CHECKLIST

- ☑ Use the correct IngressClass.
- ☑ Backend Services exist in the correct namespace.
- ☑ Services have healthy Endpoints.
- ☑ Host rules match DNS records.
- ☑ Paths match the application routes.
- ☑ Controller Pods are highly available.
- ☑ TLS certificates are valid and not expiring.
- ☑ Monitor Ingress metrics and error logs.

REMEMBER
Ingress problems block traffic before it even reaches your application. Fix the gateway first.

PRO TIP: Always check DNS → Ingress → Service → Endpoints → Pods in this order.

VERIQTA
TLS CERTIFICATE FAILURE

TLS protects trust. When TLS breaks, users lose trust and traffic stops.

When TLS is broken, browsers or clients will not trust your application, even if everything else is working.

1. FAILURE SCENARIOS

EXPIRED CERTIFICATE
The certificate has passed its validity period.

WRONG HOSTNAME
The certificate does not match the requested domain.

MISSING SECRET
The TLS Secret referenced by Ingress is missing.

CERTIFICATE CHAIN FAILURE
Intermediate or root certificate is missing or invalid.

CERT-MANAGER ISSUES
cert-manager cannot issue, renew, or store certificates correctly.

ACME VALIDATION FAILURE
Domain validation fails and the certificate cannot be issued.

2. HOW TLS WORKS IN KUBERNETES



1. Client connects using HTTPS to your domain.
2. Ingress terminates TLS using the certificate in the Secret.
3. If TLS is valid, traffic is forwarded to the Service.
4. Service routes traffic to healthy Pods.

3. QUICK DIAGNOSIS COMMANDS

`kubectl get ingress -A`	Check Ingress resources and status.
`kubectl describe ingress <ingress> -n <ns>`	Check TLS, rules, backend and events.
`kubectl get secret -n <ns>`	Verify TLS Secret exists.
`kubectl describe secret <secret> -n <ns>`	Check certificate details and expiry.
`kubectl get certificaterequests -A`	Check cert-manager CertificateRequests.
`kubectl describe certificaterequest <name> -n <ns>`	Inspect validation and issuance errors.
`kubectl get challenges -A`	Check ACME challenges and status.
`kubectl logs -n cert-manager deploy/cert-manager`	Review cert-manager controller logs.

4. HOW TO VERIFY CERTIFICATE

```
# From your laptop or a debug pod
openssl s_client -connect yourdomain.com:443 -showcerts
```

CHECK FOR	EXAMPLE OUTPUT (GOOD)
☑ Certificate Expiry	☑ Verify return code: 0 (ok)
☑ Correct Hostname (CN / SAN)	☑ Certificate chain is complete
☑ Valid Certificate Chain	☑ No expired certificates
☑ Trusted Root CA	

5. COMMON BROWSER ERRORS

• NET::ERR_CERT_DATE_INVALID	Expired Certificate
• NET::ERR_CERT_COMMON_NAME_INVALID	Hostname Mismatch
• ERR_CERT_AUTHORITY_INVALID	Untrusted / Invalid CA
• ERR_SSL_PROTOCOL_ERROR	TLS Version / Config
• ERR_CERTIFICATE_TRANSPARENCY_REQUIRED	CT Log / Policy Issue

6. PRODUCTION CHECKLIST

- ☑ Certificates are valid and not expired
- ☑ Hostname matches the domain (SAN)
- ☑ TLS Secret exists in the correct namespace
- ☑ Certificate chain is complete and trusted
- ☑ cert-manager is healthy and issuing certs
- ☑ ACME challenges are completing successfully
- ☑ Ingress uses the correct TLS Secret
- ☑ Monitor certificate expiry and renew early

SECURITY REMINDER
TLS failures often appear as application outages. Always verify TLS first before debugging the app.

TLS is the foundation of trust. Protect it, monitor it, and never let it break.

VERIQTA

NETWORKPOLICY BLOCKS PRODUCTION

NetworkPolicy controls how Pods talk to each other. One wrong rule can silently break your entire application.

When traffic is blocked, applications fail. When DNS is blocked, everything fails.

1. COMMON CAUSES

DEFAULT DENY POLICY
A default deny policy blocks all traffic unless explicitly allowed.

MISSING INGRESS RULE
No rule allows incoming traffic to the Pod from required sources.

MISSING EGRESS RULE
No rule allows outgoing traffic from the Pod to required destinations.

NAMESPACE SELECTOR MISMATCH
The rule selects the wrong namespaces or excludes the correct ones.

POD SELECTOR MISMATCH
The rule selects the wrong Pods or does not match the target Pods.

DNS ACCIDENTALLY BLOCKED
Egress rules block DNS (UDP/TCP 53), causing name resolution failures.

2. PRODUCTION WORKFLOW



FLOW EXAMPLE

App Pod (frontend) ⟶ NetworkPolicy ⟶ DB Pod (backend)

If the policy does not allow this traffic, the connection is blocked.

3. TYPICAL SYMPTOMS IN PRODUCTION

- Services are unreachable
- Timeouts when calling other services
- DNS resolution failures
- Intermittent connection errors
- Health checks failing
- Some Pods work, others do not

Remember: The app is not broken. The network policy is silently blocking traffic.

4. SENIOR INVESTIGATION COMMANDS

`kubectl get networkpolicies -A`	List all NetworkPolicies in all namespaces.
`kubectl describe networkpolicy <name> -n <ns>`	Inspect a specific NetworkPolicy.
`kubectl get pods -n <ns> -l <selector> -o wide`	Verify Pod labels and selectors.
`kubectl exec -it <pod> -- nc -zv <target-ip> <port>`	Test connectivity from Pod to target.
`kubectl exec -it <pod> -- nslookup <service>`	Test DNS resolution inside the Pod.
`kubectl get ep -n <ns>`	Check Endpoints (if empty, Service traffic fails).

5. NETWORKPOLICY EVALUATION ORDER

1. Select Pods by podSelector
2. Select Traffic by policyTypes
3. Evaluate Ingress Rules (if any)
4. Evaluate Egress Rules (if any)
5. Allow or Deny Traffic

6. PRODUCTION CHECKLIST

- ☑ Review default deny policies.
- ☑ Validate all required ingress rules.
- ☑ Validate all required egress rules.
- ☑ Confirm namespace selectors.
- ☑ Confirm pod selectors.
- ☑ Ensure DNS (UDP/TCP 53) is allowed.

Golden Rule: Start with DNS. Then validate the entire traffic path from Source to Destination.

REMEMBER NetworkPolicy does not log by default. Use logging/observability to detect blocked traffic.

PRO TIP: Always test connectivity from the source Pod using real service names and real ports.

VERIQTA

CNI PLUGIN FAILURE

No CNI = No Pod networking.
No Pod networking = No application.

CNI (Container Network Interface) is the foundation of Pod networking. When the CNI fails, Pods lose connectivity.

1. COMMON CAUSES

CNI DAEMON UNAVAILABLE
CNI pods (DaemonSets) crash, are NotReady, or cannot start.

IP ALLOCATION FAILURE
CNI cannot allocate IP addresses from the pool (IP exhaustion).

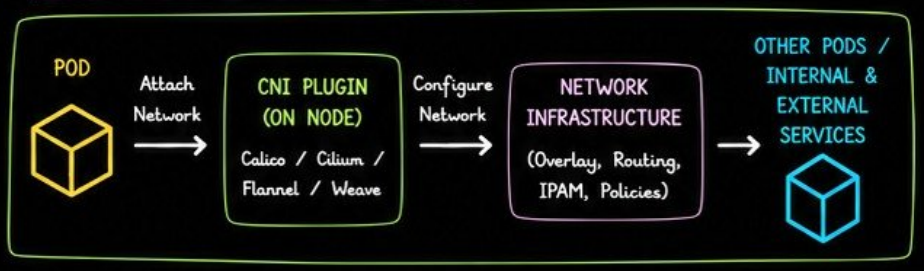
OVERLAY NETWORK PROBLEMS
VXLAN/GRE tunnel issues, MTU mismatch, or encapsulation failures.

ROUTING ISSUES
Routes to Pods are missing or incorrect on Nodes.

VXLAN FAILURE
VXLAN ports blocked, flannel/cilium tunnels down, or bad encapsulation.

PLUGIN CONFIGURATION ERRORS
Wrong config, bad IP ranges, or invalid plugin settings.

2. HOW CNI WORKS (HIGH LEVEL)



1. Pod is created on the Node.
2. Kubelet asks CNI plugin to attach network.
3. CNI configures IP, routes, and policies.
4. Pod can communicate with other Pods and services.

3. COMMON CNI PLUGIN EXAMPLES

CALICO	CILIUM	FLANNEL	WEAVE NET
• BGP routing	• eBPF based	• Simple & lightweight	• Fast & simple
• Network policies	• Advanced policies	• VXLAN overlay	• Encrypted overlay
• IP-in-IP / VXLAN	• Observability	• Easy to deploy	• Automatic encryption
• High performance	• High performance	• Good for small clusters	• Easy visualization

4. SENIOR INVESTIGATION COMMANDS

`kubectl get pods -n kube-system -o wide`	Check CNI pods (DaemonSets) status and Nodes.
`kubectl -n kube-system get ds`	List all CNI DaemonSets.
`kubectl -n kube-system logs -l k8s-app=calico-node`	Check logs for Calico (change label for other CNIs).
`kubectl get nodes -o wide`	Verify Node network status and conditions.
`ip addr`	Check Node IP addresses and interfaces.
`ip route`	Check routing table on the Node.
`ping <pod-ip>`	Test connectivity from Node to Pod IP.
`traceroute <pod-ip>`	Trace network path to Pod.

5. SYMPTOMS YOU WILL SEE

- Pods stuck in ContainerCreating
- Pods cannot talk to each other
- Services unreachable
- High network latency
- DNS failures in the cluster
- Intermittent connectivity
- Node NotReady or flapping

Most application issues are symptoms, not the real problem. Check your network first.

6. QUICK CHECK WORKFLOW

1. Check CNI DaemonSet Pods
2. Check Logs for Errors
3. Validate IPAM & Routes
4. Test Pod Connectivity End-to-End
5. Fix Root Cause & Validate Again

REMEMBER

If Pods cannot reach each other, or Services/DNS are failing, 90% of the time it's a networking problem. Start with CNI.

7. PRODUCTION CHECKLIST

- ☑ CNI DaemonSet Pods are Running on all Nodes
- ☑ No CrashLoopBackOff in CNI Pods
- ☑ Node IP allocation is healthy
- ☑ Routes are correct and stable
- ☑ Overlay network (VXLAN) is up
- ☑ NetworkPolicies are not overly restrictive
- ☑ DNS is resolving inside Pods
- ☑ Monitor CNI metrics and alerts

PRO TIP: Monitor CNI health metrics. A failing CNI will take down your entire cluster.

VERIQTA

POD-TO-POD COMMUNICATION FAILURE

Healthy Pods + Bad Network = Broken App

Bad Network = Everything Fails

When Pods cannot communicate, applications break, services time out, and the business stops.

The network is the foundation. Validate it first.

1. POSSIBLE CAUSES

CNI ISSUES
CNI plugin down, IPAM failure, overlay network broken, routing misconfigured.

FIREWALL RULES
Host firewall, cloud security groups, NACLs, or node-level rules blocking traffic.

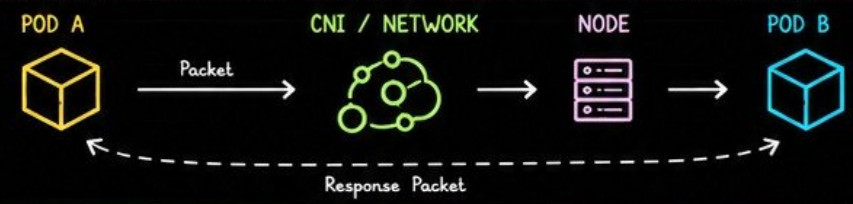
NETWORKPOLICY
Policies may be blocking ingress or egress between Pods.

ROUTING PROBLEMS
Incorrect routes, BGP/overlay issues, or asymmetric routing.

MTU MISMATCH
Packet fragmentation or MTU mismatch causing dropped packets.

IP CONFLICTS
Duplicate IP addresses cause ARP issues and intermittent connectivity.

2. HOW POD-TO-POD COMMUNICATION WORKS



All Pods in the cluster should be able to reach each other at the network layer, regardless of the node they are running on.

3. SENIOR INVESTIGATION WORKFLOW

1	**VERIFY POD STATUS** Ensure both Pods are Running and Ready.	`kubectl get pods -o wide`
2	**TEST POD-TO-POD CONNECTIVITY** From Pod A, try reaching Pod B IP and Service IP.	`kubectl exec -it <pod-a> -- ping <pod-b-ip>` `kubectl exec -it <pod-a> -- curl http://<pod-b-ip>:<port>`
3	**CHECK NETWORKPOLICIES** Ensure no policy is blocking traffic.	`kubectl get networkpolicies -A` `kubectl describe networkpolicy <name> -n <ns>`
4	**CHECK CNI HEALTH** Verify CNI pods, node agents, and IP allocation.	`kubectl -n kube-system get pods` `kubectl -n kube-system logs -l k8s-app=<cni-plugin>`
5	**CHECK ROUTING** Verify routes on Nodes and Pods (if using hostNetwork).	`ip route` `kubectl get nodes -o wide`
6	**CHECK MTU** Ensure MTU is consistent end-to-end.	`ip link` `ping <pod-ip> -s 1472 -M do`
7	**CHECK FIREWALL / CLOUD RULES** Review SGs, NACLs, host firewalls.	Verify cloud security groups Verify host firewall rules

4. COMMON SYMPTOMS

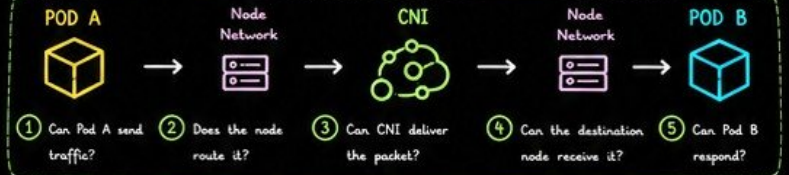
- Connection refused
- Connection timed out
- Intermittent Pod communication
- Services reachable from some Pods but not others
- High latency or packet loss
- DNS works but app traffic fails

Symptoms may look like application issues, but 80% of production outages are network issues.

5. USEFUL COMMANDS CHEAT SHEET

`kubectl get pods -o wide`	Check Pod IPs and Node location
`kubectl get svc -A -o wide`	Check Service ClusterIPs
`kubectl get endpoints <svc> -o wide`	Check Endpoint IPs
`kubectl get networkpolicies -A`	List all NetworkPolicies
`kubectl describe pod <pod> -n <ns>`	Check Pod events and IP info
`kubectl exec -it <pod> -- ip addr`	Check Pod networking interface
`kubectl logs -n kube-system -l k8s-app=<cni-plugin>`	Check CNI logs

6. NETWORK VALIDATION PATH



7. PRODUCTION MINDSET

Never assume the application is broken before validating the network.
The network is the foundation.
Fix the network first.

- ☑ Validate connectivity early
- ☑ Monitor CNI health
- ☑ Monitor latency and packet loss
- ☑ Keep MTU consistent
- ☑ Document network policies
- ☑ Test after every change

ENGINEER RULE: If Pods cannot talk to each other, nothing in the cluster can be trusted. Validate the network. Always.

EXTERNAL TRAFFIC NEVER REACHES THE CLUSTER

The traffic must successfully pass through every layer before it reaches your Pods.

When external traffic never reaches your cluster, the problem is always outside or at the edge.
The cluster is innocent until proven guilty.

1. COMMON CAUSES

LOADBALANCER PROVISIONING FAILURE
The cloud provider fails to provision or assign an external IP.

CLOUD SECURITY GROUPS
Security groups or network ACLs block incoming traffic.

FIREWALL RULES
Host firewall, cloud firewall, or WAF rules block the traffic.

INCORRECT DNS RECORDS
DNS records are wrong, missing, or pointing to the wrong IP.

INCORRECT TARGET PORTS
The Load Balancer or Ingress uses the wrong port, or the target port is wrong.

HEALTH CHECK FAILURES
Targets are marked unhealthy, so traffic is never forwarded.

2. TRAFFIC FLOW (EXTERNAL → POD)

INTERNET	LOAD BALANCER	INGRESS	SERVICE	POD
User makes a request.	Receives traffic and forwards it.	Routes traffic based on rules.	Forwards traffic to the Pods.	Pod receives the request.

If traffic fails at any step, the request never reaches the cluster workload.
Investigate from the outside in.

3. INVESTIGATION CHECKLIST (OUTSIDE → INSIDE)

1	**CHECK FROM INTERNET**
	Can you reach the Load Balancer from the internet?
		`curl -I http://<your-domain>`
		`nslookup <your-domain>`
		`dig <your-domain>`

2	**CHECK LOAD BALANCER** Is the Load Balancer active and has an external IP?	`kubectl get svc -A` (Type LoadBalancer)
3	**CHECK SECURITY GROUPS** Are inbound rules allowing traffic on the correct port?	`aws ec2 describe-security-groups` (Check inbound rules)
4	**CHECK FIREWALL / WAF** Is the firewall or WAF blocking the request?	Check cloud firewall logs Check WAF logs Check host firewall rules
5	**CHECK DNS RECORDS** Do DNS records point to the correct Load Balancer IP?	`dig <your-domain>` `nslookup <your-domain>`
6	**CHECK TARGET HEALTH** Are targets healthy? Are health checks passing?	`aws elbv2 describe-target-health` `kubectl describe svc <svc>`
7	**CHECK PORTS** Are the correct ports configured end-to-end?	`kubectl get svc <svc> -o wide` `kubectl describe ingress <ingress>`

4. RAPID VALIDATION COMMANDS

`kubectl get svc -A`	List all Services
`kubectl get ingress -A`	List all Ingresses
`kubectl describe svc <svc>`	Inspect Service events & IP
`kubectl describe ingress <ing>`	Check Ingress rules & events
`kubectl get endpoints <svc>`	Verify backend endpoints
`kubectl get pods -l <selector>`	Check Pod status
`curl -I http://<domain>`	Test from external network
`dig <domain> +short`	Resolve DNS record
`traceroute <lb-ip>`	Trace network path

5. TROUBLESHOOTING FLOW (OUTSIDE → INSIDE)



PRODUCTION REMINDER
- Always validate from the user's perspective.
- Start from the outside and go inward.
- Logs, metrics, and events are your best friends.
- Avoid making changes before you know the layer.

GOLDEN RULE
Never assume the application is broken before validating the network.
The network is the foundation.
Fix the network first.

PRO TIP
Keep a runbook, network diagrams, and access to logs.
Incidents are easier to solve when you can see everything.

VERIQTA

LOADBALANCER HEALTH CHECK FAILURE

The Load Balancer only sends traffic to targets (Pods/Nodes) that pass health checks.

The Load Balancer removes unhealthy targets. When ALL targets are unhealthy, traffic stops reaching your application.

Always validate health checks before assuming the application is down.

1. FAILURE SCENARIOS

PODS NOT READY
Pods fail readiness probe or are still starting.

WRONG HEALTH ENDPOINT
The LB checks the wrong path or the endpoint is not exposed.

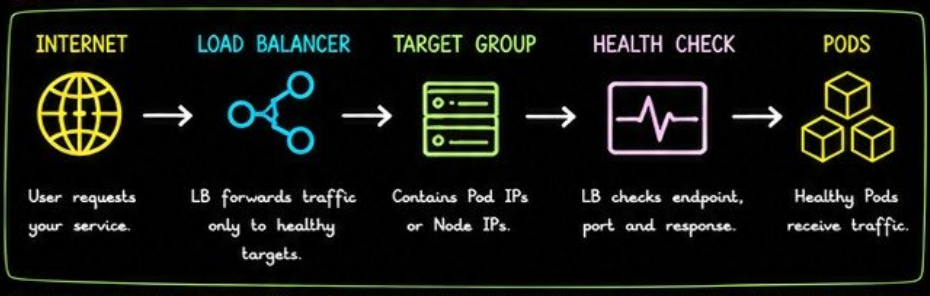
INCORRECT PORT
Health check port does not match the Service or container port.

SECURITY RESTRICTIONS
Security Groups, NACLs or firewalls block health check traffic.

SLOW STARTUP
Pods take longer to start than the health check timeout.

PROBE FAILURES
Liveness or readiness probes continuously failing.

2. LOAD BALANCER HEALTH CHECK FLOW



If a target fails, the LB stops sending traffic to it.
If ALL targets fail, users see errors.

3. HEALTH CHECK CONFIGURATION CHECKLIST

CHECK ITEM	WHAT TO VERIFY	EXAMPLE / NOTES
Health Check Path	Correct endpoint path is configured.	/healthz or /api/health Must return 200 OK
Health Check Port	Port matches the Service or container port.	Port 80 or 8080 etc.
Protocol	HTTP or HTTPS is correct.	HTTPS needs valid certificate for LB to trust.
Timeout	Timeout is long enough for your app.	Too small = false failures
Healthy / Unhealthy Thresholds	Thresholds are tuned for startup & load.	Example: healthy 3, unhealthy 3
Security Access	LB can reach the Pod from the network.	SG, NACL, Firewall, NetworkPolicy
Pod Probes	Readiness & liveness probes are correct.	If probes fail, Pod won't be Ready

4. USEFUL COMMANDS

`kubectl get pods -o wide`	Check Pod status and IPs.
`kubectl describe pod <pod>`	Check events & probe failures.
`kubectl get svc -A`	Verify Service type & ports.
`kubectl describe svc <svc>`	Check target port & endpoints.
`kubectl get endpoints <svc> -o wide`	Verify endpoint IPs & readiness.
`kubectl logs <pod>`	Check application logs.
`curl -I http://<pod-ip>:<port><path>`	Test health endpoint directly.
`nslookup <your-domain>`	Check DNS resolution.
`dig <your-domain>`	Check DNS and TTL.
`aws elbv2 describe-target-health \ --target-group-arn <arn>`	Check target health in AWS.

5. TROUBLESHOOTING WORKFLOW



6. PRODUCTION LESSONS

- ☑ Healthy Pods do not always mean healthy LB targets.
- ☑ Always test health endpoints from outside the cluster.
- ☑ Tune timeouts and thresholds for slow applications.
- ☑ Use dedicated /health endpoints (lightweight).
- ☑ Monitor target health metrics and set alerts.
- ☑ Validate security rules after every change.
- ☑ Document health check configuration.

PRODUCTION LESSON
Healthy pods do not always mean healthy load balancer targets.
Validate the entire traffic path.

SENIOR REMINDER
Production incidents are rarely caused by the application itself. They are caused by misconfigurations, assumptions, and overlooking the network.

PRO TIP
Always have a runbook for LB health check failures. It saves time, reduces stress, and protects your users.

VERIQTA

SESSION AFFINITY PROBLEMS

Session affinity (sticky sessions) ensures that a user keeps going to the same backend.

When it breaks, users see random logouts, errors, and inconsistent state.

When session affinity fails, users appear to be randomly logged out or see inconsistent data because their requests hit different Pods.

1. COMMON CAUSES

STICKY SESSION MISCONFIGURATION
Affinity not enabled, wrong type, wrong duration, or not set on the correct layer.

CACHE INCONSISTENCY
Different Pods have different cache state or sessions stored only in memory.

STATEFUL APPLICATIONS
Applications store session state locally instead of in a shared store.

SESSION REPLICATION FAILURE
Session store replication lag, cluster partition, or node failure.

COOKIE CONFIGURATION
Cookie domain, path, Secure, HttpOnly, or SameSite settings are incorrect.

LOAD BALANCING BEHAVIOUR
Load balancer rebalances aggressively, or affinity not supported at the layer you are using.

2. SESSION AFFINITY FLOW

USER	LOAD BALANCER	POD A	SESSION STORE	RESPONSE
User sends request.	LB uses affinity (cookie or source IP) to select the same backend.	Request goes to the same Pod.	Session data is read from shared store.	Consistent response returned to the user.

If affinity breaks, the next request may go to a different Pod that does not have the session state.

3. HOW IT BREAKS (EXAMPLES)

CAUSE	WHAT HAPPENS	USER IMPACT	EXAMPLE / NOTE
Sticky session misconfiguration	Requests are sent to different Pods.	Random logouts, inconsistent cart, form resets.	Affinity not enabled, wrong cookie name, wrong duration.
Cache inconsistency	Different Pod has different state.	Old data, missing items, inconsistent UI.	Local cache only, no shared cache.
Stateful applications	Session stored in Pod memory.	State lost when Pod changes.	Pod restart, scaling, rolling update.
Session replication failure	Session store not replicating properly.	Login loop, expired sessions.	Redis cluster failover, network partition.
Cookie configuration	Cookie not sent or not recognized.	Affinity not applied, new sessions created.	Domain/path mismatth, SameSite issues.
Load balancing behaviour	LB overrides or ignores affinity.	Traffic jumps between Pods.	Cross-zone LB, health check routes.

4. USEFUL COMMANDS

`kubectl get svc -A`	List all Services
`kubectl describe svc <svc>`	Check session affinity config
`kubectl get endpoints <svc> -o wide`	See endpoint distribution
`kubectl get pods -l app=<app> -o wide`	Check Pod IPs and nodes
`curl -I --cookie "<cookie>" <url>`	Test cookie-based affinity
`kubectl logs -n kube-system <coredns>`	Check DNS issues affecting cookies
`redis-cli info replication`	Check session store replication

5. INVESTIGATION CHECKLIST

- [x] Verify session affinity is enabled and configured correctly.
- [x] Confirm the correct affinity type (cookie / source IP).
- [x] Check cookie domain, path, Secure, and SameSite settings.
- [x] Ensure all Pods can access the session store.
- [x] Check cache consistency across all Pods.
- [x] Validate that session store replication is healthy.
- [x] Review load balancer behaviour and health checks.
- [x] Test with the same cookie across multiple requests.

6. TROUBLESHOOTING WORKFLOW

1. Confirm the problem (user reports, logs, metrics)
2. Check Service affinity configuration
3. Inspect cookie settings and behaviour
4. Verify endpoint distribution (Pods)
5. Check session store health & replication
6. Validate network & security rules
7. Test with the same session multiple times

PRO TIP: Use a centralized session store (Redis, DynamoDB, Memcached, DB) and configure affinity at the appropriate layer for your architecture.

7. PRODUCTION LESSON

Healthy pods do not always mean healthy load balancer targets or stable sessions.

Validate the entire session path, not just the Pods.

VERIQTA

API GATEWAY AND INGRESS FAILURES

Gateways and Ingress are the front door to your application. When the door fails, users never reach your Pods.

Gateway and Ingress failures prevent requests from reaching your Services. Most issues occur before traffic hits your Pods. Always validate each layer of the request path.

1. COMMON CAUSES

BACKEND TIMEOUT
The backend takes too long to respond within the allowed timeout.

502 BAD GATEWAY
The gateway received an invalid response from the upstream server.

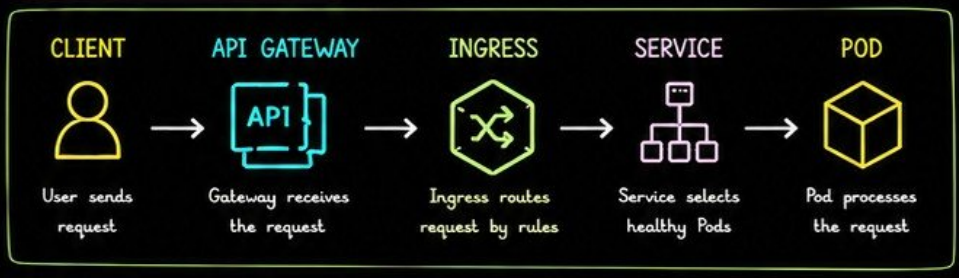
503 SERVICE UNAVAILABLE
The upstream service is down, overloaded, or not available.

504 GATEWAY TIMEOUT
The gateway did not receive a response in time from upstream.

CONNECTION RESET
The connection was closed unexpectedly by the backend or a network device.

BACKEND UNAVAILABLE
The backend Service, Pods, or endpoints are not reachable.

2. PRODUCTION WORKFLOW



A failure at any layer stops the request.
The user sees an error even if the application is healthy.

3. ERROR CODE QUICK REFERENCE

ERROR	WHAT IT MEANS	COMMON CAUSES	WHERE IT OCCURS	USER IMPACT
502 Bad Gateway	Gateway received an invalid response from upstream.	• Backend returned invalid response • Crash or misconfiguration • Connection closed early	API Gateway or Ingress	Users get 502 Bad Gateway
503 Service Unavailable	Service is down or unable to handle requests.	• Pods not ready • No healthy endpoints • Overloaded service	API Gateway or Ingress	Users get 503 Service Unavailable
504 Gateway Timeout	Gateway did not receive a response in time.	• Backend timeout • Slow application • Long processing time	API Gateway or Ingress	Users get 504 Gateway Timeout
Connection Reset	Connection closed before response was completed.	• Backend reset connection • Firewall/NAT timeout • Network instability	Network path or backend	Users see connection error or retry loop

4. INVESTIGATION CHECKLIST

- ☑ Check Gateway logs (CloudWatch / Stackdriver / App Logs).
- ☑ Check Ingress controller logs.
- ☑ Verify Service and Endpoints are healthy.
- ☑ Confirm Pod readiness and liveness.
- ☑ Test backend directly from inside the cluster.
- ☑ Validate DNS and host rules.
- ☑ Check security groups, NACLs, and firewalls.
- ☑ Review timeouts and limits.
- ☑ Reproduce the issue with curl or Postman.
- ☑ Document findings before making changes.

5. LAYER-BY-LAYER VALIDATION COMMANDS

LAYER	WHAT TO CHECK	COMMANDS / EXAMPLES	EXPECTED RESULT
API GATEWAY	Gateway health and backend integration	`aws apigateway get-stages` Check integration latency & errors	Gateway is healthy, no 5XX errors
INGRESS	Ingress controller status and events	`kubectl get ingress -A` `kubectl logs -n ingress-nginx deploy/ingress-nginx-controller`	Ingress rules loaded correctly, no errors
SERVICE	Service has healthy endpoints	`kubectl get svc -o wide` `kubectl get endpoints <svc-name> -o wide`	Endpoints list shows healthy Pod IPs
POD	Pods are Ready and responding	`kubectl get pods -o wide` `kubectl describe pod <pod-name>`	Pods are Running and Ready

6. TROUBLESHOOTING FLOW



7. PRODUCTION LESSON

Users don't care where the failure happened. They only see the error.
Validate every layer of the path, not just the Pods.
Think: Gateway → Ingress → Service → Pod

PRO TIP: Always monitor latency, error rate, and success rate at every layer. A small delay today can become an outage tomorrow.

VERIQTA

SERVICE MESH FAILURE

Service mesh adds power, observability, and security to your applications — but when it breaks, everything can stop talking.

A Service Mesh manages how services communicate. When the mesh fails, requests may be blocked, delayed, or misrouted.
Always validate the mesh before blaming the application.

1. WHAT IS A SERVICE MESH?

A Service Mesh is a dedicated infrastructure layer that controls service-to-service communication.

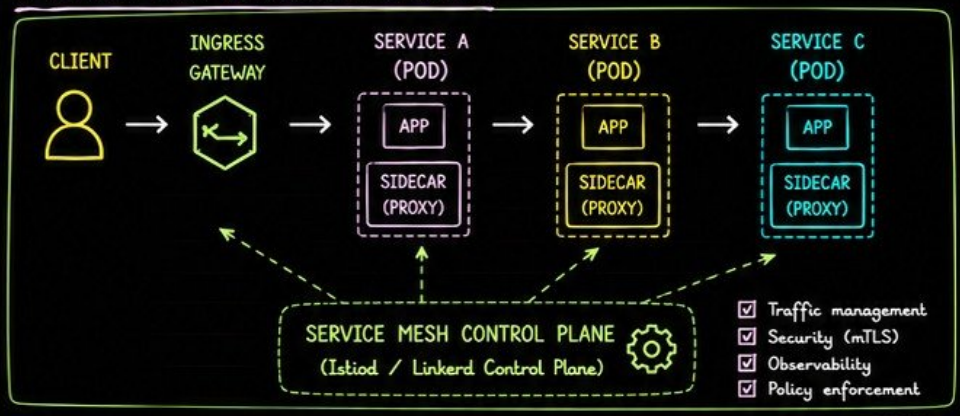
EXAMPLES

Istio
- Rich features
- Strong observability
- Enterprise ready

Linkerd
- Lightweight
- Simple
- Performance focused

Both inject sidecars (proxies) into Pods to manage traffic.

2. SERVICE MESH ARCHITECTURE OVERVIEW



If the sidecar, policy, or control plane fails, communication is affected.

3. COMMON SERVICE MESH FAILURES

SIDECAR INJECTION FAILURE
Sidecar proxy is not injected or is not running in the Pod.

mTLS PROBLEMS
Mutual TLS handshake fails between services.

AUTHORIZATIONPOLICY ISSUES
Policies are too strict or misconfigured, blocking legitimate traffic.

VIRTUALSERVICE ERRORS
Routing rules are wrong, missing, or conflicting.

DESTINATIONRULE MISTAKES
Incorrect subsets, traffic policies, or load balancing settings.

CERTIFICATE ROTATION ISSUES
Expired or untrusted certificates break mTLS communication.

4. FAILURE DETAILS AND SYMPTOMS

FAILURE	WHAT HAPPENS	COMMON CAUSES	SYMPTOMS	CHECK THIS
Sidecar Injection Failure	Pod runs without proxy, traffic bypasses the mesh.	• Missing/incorrect label • Webhook failure • Namespace exclusion	• Pod not in mesh • No sidecar container • Traffic is unencrypted	kubectl get pods -o wide Look for sidecar container Check namespace labels
mTLS Problems	Secure connection cannot be established.	• Cert expired • Wrong SAN/CN • Clock skew • Trust issue	• 503 UF / 495 SSL • mTLS handshake fail • Connection reset	istioctl x authn tls-check Check certs & validity Verify time sync (NTP)
AuthorizationPolicy Issues	Requests are denied by policy rules.	• Deny rule too broad • Wrong selector • Missing allow rule	• 403 Forbiddon • RBAC-like denials • No response	istioctl x authz check Review policies Check source labels
VirtualService Errors	Traffic routed to the wrong destination or no destination.	• Wrong host/name • Bad match rules • Conflicting VS	• 404 / 503 errors • Traffic not routed • Unexpected endpoints	kubectl analyze Check VS routes Validate host & paths
DestinationRule Mistakes	Traffic policy or subset configuration is wrong.	• Wrong subset labels • Bad LB policy • Ineffective settings	• Traffic imbalance • Retries/timeouts • No healthy upstream	istioctl proxy-config clusters <pod> Check subsets & LB
Certificate Rotation Issues	New certs not trusted or not updated in time.	• Expired certs • Secret not updated • CA bundle not trusted	• mTLS failures • 503 UF errors • Spike in resets	Check cert-manager logs Check Secret expiry Verify CA bundles

5. DEBUGGING COMMANDS CHEAT SHEET

`istioctl proxy-status`	Check sidecar connectivity
`istioctl proxy-config clusters <pod>`	List clusters and endpoints
`istioctl proxy-config routes <pod>`	List routes (VirtualService)
`istioctl proxy-config listeners <pod>`	Check listeners and ports
`istioctl proxy-config secret <pod>`	Check certs and secrets
`istioctl x authz check <pod> --src <src> --dst <dst>`	Validate authorization
`kubectl logs <pod> -c istio-proxy`	Check sidecar logs
`istioctl analyze`	Analyze mesh configuration

PRO TIP: When traffic breaks, think mesh first. Healthy apps still fail if the mesh blocks the path.

6. SERVICE MESH TROUBLESHOOTING FLOW

1. Observe Symptoms
2. Check Pod & Sidecar
3. Check Policies (AuthZ)
4. Check Routing (VS)
5. Check Traffic Policy (DR)
6. Check mTLS & Certs
7. Verify & Monitor

7. PRODUCTION LESSON

- ☑ The mesh is part of the data plane — treat it like production code.
- ☑ Validate sidecar injection before deploying workloads.
- ☑ Monitor mTLS, policies, and config changes continuously.
- ☑ Test routing and policies in staging before production.
- ☑ Small mesh misconfigurations can cause big outages.

CROSS-NAMESPACE COMMUNICATION FAILURE

Namespaces isolate resources by default. Cross-namespace communication must be configured correctly at multiple layers.

Applications in different namespaces must be allowed, addressed, and routed correctly. When any part is wrong, the connection fails.
Namespace isolation is powerful — and easy to misconfigure.

1. COMMON CAUSES

WRONG DNS NAME
Using the wrong Service name, namespace, or DNS format.

RBAC CONFUSION
ServiceAccount in one namespace lacks permissions to access resources in another.

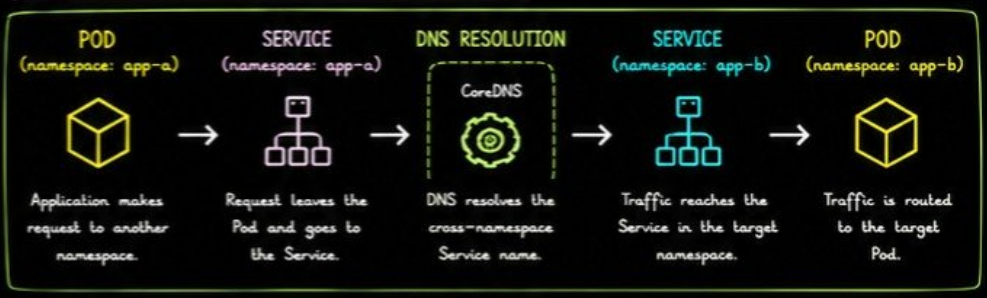
NETWORKPOLICY
Default deny or missing rules block traffic between namespaces.

NAMESPACE ISOLATION
Resources are isolated by default. No rule or export configured.

SERVICE EXPORT ISSUES
Service not exposed, not exported, or using wrong visibility (ClusterIP only).

INCORRECT FQDN USAGE
Wrong or incomplete FQDN in code, config, or environment variables.

2. CROSS-NAMESPACE COMMUNICATION FLOW



If any step fails, the communication breaks.
Validate every layer: DNS → Network → RBAC → Service → Pod

3. CAUSE, IMPACT, AND HOW TO CHECK

CAUSE	IMPACT	COMMON SIGNS	HOW TO CHECK
Wrong DNS Name	Service not found	lookup <svc> no such host Temporary failure in name resolution	nslookup <svc>.<ns>.svc.cluster.local kubectl get svc -n <ns>
RBAC Confusion	403 Forbidden or access denied	Forbidden: User cannot access resource in the namespace	kubectl auth can-i --as=<sa> get pods -n <ns> kubectl get rolebinding -n <ns>
NetworkPolicy	Connection timeout No response	i/o timeout Connection refused	kubectl get networkpolicy -A kubectl describe networkpolicy <name> -n <ns>
Namespace Isolation	Traffic blocked even with correct DNS	Pods cannot reach each other across namespaces	kubectl get networkpolicy -A Check default deny rules
Service Export Issues	Service not reachable outside namespace	No endpoints Service not found	kubectl get endpoints -n <ns> kubectl get svc -n <ns>
Incorrect FQDN Usage	Request fails or goes to wrong service	404 / Wrong service Wrong backend hit	Verify FQDN format in config <svc>.<namespace>.svc.cluster.local

4. USEFUL COMMANDS

nslookup <svc>.<ns>.svc.cluster.local	Test DNS resolution
kubectl get svc -A	List all Services
kubectl get endpoints -A	Check Service endpoints
kubectl get pods -A	List all Pods
kubectl get networkpolicy -A	List NetworkPolicies
kubectl describe networkpolicy <name> -n <ns>	Inspect policy
kubectl auth can-i --as=<sa> get pods -n <ns>	Check RBAC access
curl http://<svc>.<ns>.svc.cluster.local:port	Test service connectivity

5. CORRECT DNS FORMATS

☑ <svc>.<namespace>.svc.cluster.local	Fully Qualified Domain Name (FQDN)
☑ <svc>.<namespace>.svc	Short FQDN (recommended)
☑ <svc>.<namespace>	Bare format (works within cluster)
☑ <svc>	Only works within same namespace

Always prefer the full or short FQDN for clarity and reliability.

6. INVESTIGATION WORKFLOW



7. CROSS-NAMESPACE CHECKLIST

- ☑ DNS name is correct and resolvable
- ☑ Target Service exists in the right namespace
- ☑ Service has healthy Endpoints
- ☑ NetworkPolicy allows traffic
- ☑ RBAC permissions are correct
- ☑ FQDN used in app configuration is accurate
- ☑ No default deny blocking the traffic
- ☑ Test connectivity from source Pod
- ☑ Application logs show successful requests

8. PRODUCTION LESSON

- ☑ Namespaces isolate by default — design with intent.
- ☑ Always validate DNS, Network, RBAC, and Endpoints before debugging the app.
- ☑ Use FQDN consistently in configurations and environment variables.
- ☑ Document allowed cross-namespace communication.
- ☑ Monitor and alert on connection failures and policy changes.

PRO TIP
Use namespace labels + NetworkPolicies for clean, predictable communication between teams and services.

VERIQTA

MULTI-CLUSTER TRAFFIC FAILURE

Multi-cluster systems add complexity. A failure in routing, discovery, or failover can make services unreachable across environments or regions. Design for failure. Test failover. Monitor continuously.

Multi-cluster means multiple Kubernetes clusters working together. When traffic stops flowing, the whole business can stop.

1. MULTI-CLUSTER NETWORKING

WHAT IT IS
Clusters in different environments, regions, or clouds that need to communicate securely.

COMMON PATTERNS
- Hub-and-Spoke
- Mesh (Full Connectivity)
- Federation

NETWORK TECHNOLOGIES
- VPC Peering / Transit Gateway
- VPN / Direct Connect
- Service Mesh (Istio / Linkerd)
- Overlay Networks (VXLAN, WireGuard)

2. COMMON TRAFFIC FAILURE CAUSES

GLOBAL LOAD BALANCER ISSUES
Health checks fail, wrong region selected, or misconfiguration.

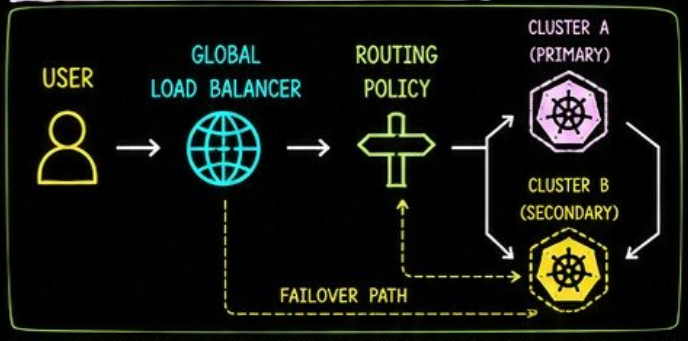
CROSS-REGION ROUTING PROBLEMS
Latency, routing policies, or network ACLs block traffic.

SERVICE DISCOVERY FAILURES
DNS resolution issues, stale records, or wrong endpoints.

FAILOVER NOT TRIGGERED
Health checks fail to detect issue or failover rules misconfigured.

DISASTER RECOVERY GAPS
No backups, no replication, or RPO/RTO not met.

3. MULTI-CLUSTER TRAFFIC FLOW (EXAMPLE)



If the primary path fails, traffic must automatically shift to the healthy cluster.

4. KEY COMPONENTS AND THEIR ROLE

COMPONENT	ROLE	EXAMPLES
Global Load Balancer	Entry point for global traffic. Distributes traffic based on health, latency, or policy.	AWS Global Accelerator Cloudflare Load Balancing Google Cloud Global LB
Routing Policy	Decides which cluster/region should receive traffic.	Latency-based routing Health-based routing Geolocation routing
Service Discovery	Resolves service names to the correct endpoints.	CoreDNS Consul AWS Cloud Map
Failover Mechanism	Detects failure and shifts traffic automatically.	Health checks Failover rules Outlier detection
Disaster Recovery	Ensures business continuity during major outages.	Cross-region replication Backups DR runbooks

5. FAILOVER STRATEGIES

ACTIVE-ACTIVE
Traffic split across multiple clusters. High availability.

ACTIVE-PASSIVE
One cluster handles traffic, other stands by.

ACTIVE-ACTIVE WITH FAILOVER
Both active, but one handles more traffic. Failover on failure.

REGION FAILOVER
Traffic routed to a secondary region on outage.

6. INVESTIGATION CHECKLIST

- [x] Check Global Load Balancer health and target status.
- [x] Verify routing policy and endpoint selection.
- [x] Test DNS resolution from multiple regions.
- [x] Verify network connectivity between clusters.
- [x] Check Service Discovery endpoints.
- [x] Review Failover rules and triggers.
- [x] Inspect application health on both clusters.
- [x] Check logs, metrics, and alerts.

7. TROUBLESHOOTING COMMANDS CHEAT SHEET

`dig <service>.<namespace>.svc.cluster.local`	Test DNS resolution
`kubectl get endpoints -A`	Check service endpoints
`kubectl get svc -A`	Check Services and IPs
`kubectl get pods -A -o wide`	Check Pod locations
`kubectl describe svc <svc>`	Inspect Service configuration
`kubectl logs -n <ns> <pod>`	Check application logs
`curl -I <service-endpoint>`	Test connectivity
`mtr <destination-ip>`	Trace route and latency
`istioctl proxy-status`	Check Mesh proxy status (if using Istio)

8. BEST PRACTICES

- [x] Design for failure from day one.
- [x] Use health-based routing.
- [x] Keep clusters in sync.
- [x] Automate failover and testing.
- [x] Monitor latency and errors globally.
- [x] Use service mesh for observability and traffic control.
- [x] Document DR plan and test regularly.

9. PRODUCTION WORKFLOW SUMMARY

1. User sends request
2. Global LB receives traffic
3. Routing policy selects cluster
4. Traffic sent to primary cluster
5. Health checks monitor cluster
6. If failure detected
7. Failover to healthy cluster
8. Service remains available

10. PRODUCTION LESSON

Multi-cluster is not just networking. It is a strategy for resilience.

People trust systems that stay up — even when things break. Build systems that heal.

PRO TIP: Test your failover in staging before you need it in production. A failover you didn't test will fail you.

VERIQTA

ENTERPRISE NETWORKING INVESTIGATION WORKFLOW

A structured workflow eliminates guesswork.
Follow the path.
Validate each layer.
Fix the root cause, not just the symptom.

Production incidents are rarely random.
Follow the workflow. Investigate layer by layer. Solve permanently.

1. ENTERPRISE NETWORKING INVESTIGATION PATH

1. **USER IMPACT** — Users cannot access application
2. **DNS** — Domain resolution check
3. **LOAD BALANCER** — Traffic reaches Load Balancer successfully?
4. **INGRESS** — Ingress rules routing traffic correctly?
5. **SERVICE** — Service routes traffic to correct Pods?
6. **ENDPOINTS** — Endpoints available and healthy?
7. **NETWORKPOLICY** — Policy allows traffic between components?
8. **POD** — Pod Ready? Running? Receiving traffic?
9. **APPLICATION** — Application responding correctly?

Start at the left. Move right. Validate each layer before going deeper.

2. RECOVERY PRIORITIES (DO THIS FIRST)

1. **RESTORE TRAFFIC**
   - Get users back online as fast as possible.
   - Focus on availability first.
2. **VALIDATE ROUTING**
   - Ensure traffic flows through each layer correctly.
   - Check routes, rules, targets.
3. **CONFIRM SERVICE DISCOVERY**
   - Validate DNS, Services, Endpoints.
   - Ensure components can find each other.
4. **MONITOR RECOVERY**
   - Watch metrics, logs, and alerts.
   - Ensure stability and performance.

3. LAYER CHECKS – WHAT TO LOOK FOR

LAYER	WHAT TO CHECK	COMMON ISSUES	TOOLS / COMMANDS
USER IMPACT	Verify real user impact and scope.	• Intermittent vs total outage • Specific region / users	• User reports • Monitoring dashboards
DNS	DNS resolves to correct Load Balancer IP.	• Wrong DNS record • TTL caching issues	• nslookup / dig • Route 53 / cloud DNS
LOAD BALANCER	LB is healthy and receiving traffic.	• Health check failures • Wrong target group	• cloud console • curl / nc / traceroute
INGRESS	Ingress controller is routing requests.	• Wrong host/path rules • SSL/TLS issues	• kubectl get ingress • kubectl logs -n ingress-nginx
SERVICE	Service selects correct Pods.	• Wrong selector • Wrong port	• kubectl get svc • kubectl describe svc
ENDPOINTS	Endpoints are present and Ready.	• No endpoints • Unhealthy Pods	• kubectl get endpoints • kubectl get ep -o wide
NETWORKPOLICY	Policies allow required traffic.	• Default deny blocking • Missing rules	• kubectl get networkpolicy • kubectl describe netpol
POD	Pod is Running and Ready.	• CrashLoopBackOff • Image pull errors	• kubectl get pods -o wide • kubectl describe pod
APPLICATION	Application is healthy and responding.	• App errors • Dependancy failures	• curl / http • App logs / APM

4. INVESTIGATION COMMANDS CHEAT SHEET

`kubectl get pods -A -o wide`	List all Pods across namespaces
`kubectl get svc -A -o wide`	List all Services across namespaces
`kubectl get ingress -A`	List all Ingress resources
`kubectl get endpoints -A -o wide`	Check Endpoints
`kubectl get networkpolicy -A`	List all NetworkPolicies
`kubectl describe pod <pod>`	Detailed Pod info and events
`kubectl logs <pod> -n <ns>`	View Pod logs
`kubectl exec -it <pod> -- sh`	Access Pod shell
`nslookup <domain>`	DNS resolution check
`curl -I http://<domain>`	HTTP connectivity test
`traceroute <domain>`	Network path check

5. INVESTIGATION WORKFLOW (STEP BY STEP)

1. Identify User Impact
2. Check DNS Resolution
3. Verify Load Balancer Health
4. Validate Ingress Routing
5. Check Service Routing
6. Verify Endpoints Health
7. Validate NetworkPolicy Rules
8. Verify Pod & App Health

Stop at the layer where it breaks. Fix there. Then re-validate the path.

6. QUICK DECISION GUIDE

SYMPTOM	START HERE	LIKELY LAYER
Cannot access domain	DNS	DNS / DNS Records
Connection timeout	Load Balancer	LB / Network / SG
404 or 503 errors	Ingress	Ingress / Routing
No endpoints	Service	Service / Selector
Traffic blocked	NetworkPolicy	Policy / Firewall
Pod not reachable	Pod	Pod / Container
App errors	Application	Application / Dependencies

7. PRODUCTION TIPS

- [x] Always start with the user impact.
- [x] Validate each layer before moving forward.
- [x] Use metrics + logs + traces together.
- [x] Document findings and actions.
- [x] Automate checks with monitoring.
- [x] Practice failure drills and game days.

8. PRODUCTION MINDSET

Slow is smooth.
Smooth is fast.

Investigate with structure.
Resolve with precision.
Prevent with learning.

PRO TIP: Most outages are not random.
They follow patterns. Learn them. Break them. Prevent them.

VERIQTA

PRODUCTION NETWORKING INCIDENT PLAYBOOK

When traffic breaks,
follow the playbook.
Do not guess.
Follow the flow.
Validate each layer.
Recover with confidence.

This playbook is for production incidents where users cannot reach the application or experience errors.
Follow the steps in order. Validate before moving forward.

1. INCIDENT FLOW (FOLLOW IN ORDER)

1. **ALERT**
   Receive alert from monitoring system or user report.
2. **VERIFY USER IMPACT**
   Confirm real user impact. Check scope and severity.
3. **VALIDATE DNS**
   Check DNS resolution, TTL, and records.
4. **INSPECT LOAD BALANCER**
   Check LB health, targets, listeners, and SG rules.
5. **INSPECT INGRESS**
   Validate Ingress rules, paths, host, and TLS.
6. **VALIDATE SERVICES**
   Check Service type, selector, and ports.
7. **CHECK ENDPOINTS**
   Ensure endpoints exist and are Ready.
8. **TEST POD CONNECTIVITY**
   Test network connectivity to Pods and application.
9. **RECOVER TRAFFIC**
   Apply fix, reload configs, or restart components.
10. **MONITOR**
    Monitor metrics, logs, and user experience.

2. QUICK DIAGNOSTIC COMMANDS BY LAYER

LAYER	WHAT TO CHECK	COMMANDS / EXAMPLES

DNS	DNS resolution and records	`nslookup app.example.com`
		`dig app.example.com +short`
		`kubectl get svc -A | grep external`

LOAD BALANCER	LB status, targets, listeners	`aws elbv2 describe-target-health --target-group-arn <arn>`
		`aws elbv2 describe-load-balancers`
		`kubectl get svc -A | grep LoadBalancer`

INGRESS	Ingress rules and controllers	`kubectl get ingress -A`
		`kubectl describe ingress <name> -n <ns>`
		`kubectl logs -n ingress-nginx deploy/ingress-nginx-controller`

SERVICE	Service configuration and port mapping	`kubectl get svc -A`
		`kubectl describe svc <name> -n <ns>`
		`kubectl get svc <name> -o yaml -n <ns>`

ENDPOINTS	Endpoints exist and are Ready	`kubectl get endpoints <name> -n <ns>`
		`kubectl describe endpoints <name> -n <ns>`
		`kubectl get pods -l <selector> -n <ns>`

NETWORKPOLICY	Policies affecting traffic	`kubectl get networkpolicy -A` `kubectl describe networkpolicy <name> -n <ns>`

POD	Pod health and status	`kubectl get pods -n <ns>`
		`kubectl describe pod <pod> -n <ns>`
		`kubectl logs <pod> -n <ns> --tail=100`

APPLICATION	App health and dependencies	`curl -I http://<service>`
		`curl http://<service>/healthz`
		Check app logs and APM

3. COMMON ROOT CAUSES

- DNS misconfiguration
- LB targets unhealthy
- Ingress rule incorrect
- Service selector mismatch
- No or unhealthy endpoints
- NetworkPolicy blocking traffic
- Pod not Ready or crashing
- Application not responding

4. RECOVERY PRIORITIES

1. **Restore Traffic** — Get users back online first. Use rollback or last known good.
2. **Validate Routing** — Ensure traffic flows through each layer correctly.
3. **Confirm Service Discovery** — Verify DNS, Services, and Endpoints.
4. **Monitor Recovery** — Watch metrics, logs, and user experience.

5. INCIDENT COMMUNICATION CHECKLIST

- [x] Acknowledge incident quickly.
- [x] Update status regularly.
- [x] Share impact, scope, and ETA.
- [x] Communicate actions taken.
- [x] Announce resolution.
- [x] Share root cause (when known).
- [x] Document lessons learned.

6. ESCALATION GUIDE

SEVERITY	ACTION
SEV-1	Page on-call + Leadership War room activated
SEV-2	Notify on-call team Increase monitoring
SEV-3	Create ticket Investigate
SEV-4	Routine issue Track and improve

PRO TIP: Cnaal aorck calea repoilon oncoliaalilt tg Pods ih tayer ctach ahang appng eaigpuenk tbe, caoct.

VERIQTA

VOLUME 2
PRODUCTION CHECKLIST
KUBERNETES NETWORKING

> Great engineers prepare before problems happen. Checklists save downtime.

A strong checklist prevents incidents.
Validate before you deploy. Validate during incidents.
Check everything. Assume nothing.

1. BEFORE DEPLOYMENT CHECKLIST

1. **VALIDATE SERVICES** — Ensure Services exist, ports are correct, and endpoints are populated. ☑
2. **VERIFY LABELS AND SELECTORS** — Confirm Pod labels match Service selectors exactly. ☑
3. **TEST DNS RESOLUTION** — Verify DNS records resolve to the correct LoadBalancer or endpoint. ☑
4. **VALIDATE INGRESS RULES** — Check host rules, paths, backend services, and annotations. ☑
5. **TEST TLS CERTIFICATES** — Validate certificate chain, expiration, and TLS configuration. ☑
6. **REVIEW NETWORKPOLICIES** — Ensure policies allow required traffic and do not block legitimate flows. ☑
7. **CONFIRM CNI HEALTH** — Check CNI pods running, no errors, and node network connectivity. ☑
8. **VERIFY LOADBALANCER CONFIGURATION** — Check scheme, listeners, target groups, health checks, and security groups. ☑

2. DURING AN OUTAGE CHECKLIST

1. **VERIFY DNS FIRST** — DNS issues affect everything. Check resolution, TTL, and records. ☑
2. **INSPECT ENDPOINTS** — Ensure endpoints exist and are Ready. No endpoints = no traffic. ☑
3. **VALIDATE INGRESS** — Check Ingress status, backend, rules, and controller logs. ☑
4. **CONFIRM NETWORKPOLICIES** — Policies may be blocking traffic. Validate allow rules. ☑
5. **CHECK SERVICE SELECTORS** — Ensure selectors match current Pod labels. ☑
6. **TEST POD CONNECTIVITY** — Test from Pod to Pod, Service, and external endpoints. ☑
7. **CAPTURE EVIDENCE** — Collect logs, events, configs, screenshots, and traces. ☑
8. **DOCUMENT THE INCIDENT** — Record timeline, impact, root cause, actions taken, and lessons. ☑

3. FINAL PRINCIPLE

Networking failures often look like application failures.
Senior engineers validate the complete traffic path,
from the client to the container,
before concluding that the application itself is at fault.



PRO TIP: Test everything in staging exactly as it will run in production. Your checklist is your first line of defense.

CHECK EARLY.
CHECK OFTEN.
PREVENT OUTAGES.